

PDF 문서로 RAG 구현하기

학습 체크리스트 (1/2)

☐ RAG가 필요한 이유와 파인튜닝·프롬프트 방식과의 차이 이해

☐ 문서·청크·임베딩·벡터 스토어 용어와 전체 파이프라인 파악

☐ 임베딩 차원과 pgvector HNSW 2,000차원 제한의 관계 이해

☐ Spring AI 의존성·pgvector·임베딩 모델 설정 방법 습득

학습 체크리스트 (2/2)

☐ ETL 파이프라인으로 PDF를 청크로 나누어 저장하는 흐름 습득

☐ PDF 검증·중복 업로드·트랜잭션 분리 주의점 파악

☐ `similaritySearch`와 메타데이터 필터로 검색 결과를 점검하는 방법 습득

☐ `QuestionAnswerAdvisor` 기반 답변 생성과 모델 fallback 설계 이해

LLM이 모르는 것

- 학습 시점 이후 자료: 최신 뉴스·정책·통계를 알지 못함
- 조직 내부 문서: 회사·기관 고유의 비공개 자료를 알지 못함
- 환각(hallucination): 모르는 내용을 그럴듯하게 지어냄
- 근거 부재: 답변이 어디서 나왔는지 제시하지 못함

세 가지 대응 방법 비교

방법	비용	최신성	근거 제시
파인튜닝	학습 비용 큼	다시 학습해야 반영	근거 문장 없음
프롬프트에 전문 붙여넣기	요청마다 토큰 비용 증가	붙여넣은 만큼 즉시	있으나 비효율
RAG	임베딩·검색 비용만 추가	문서 추가 시 바로 반영	검색된 문단이 근거

- 선택 기준: 비용 대비 최신성과 근거 확보가 중요할수록 RAG 유리
- 이번 차시 방향: RAG 방식으로 구현

RAG 세 글자의 의미

검색 증강 생성 (RAG, Retrieval-Augmented Generation)

질문과 관련된 문단만 찾아(Retrieval) 프롬프트에 넣고(Augmented) 답을 생성(Generation)하는 방식

- **Retrieval:** 질문과 관련된 문단을 검색으로 찾음
- **Augmented:** 찾은 문단을 프롬프트에 추가함
- **Generation:** 추가된 문단을 근거로 LLM이 답변 생성

이번 차시에서 만들 것

- 업로드 흐름 확장: Thymeleaf 폼으로 PDF 업로드 → 텍스트 추출
- 벡터화 처리: 청크 분할 → 임베딩 → pgvector 저장
- 저장 대상 변화: 404차시는 "파일"을, 이번엔 "벡터"를 저장
- 실습 소재: 정책브리핑 보도자료 PDF로 근거 있는 답변 구현

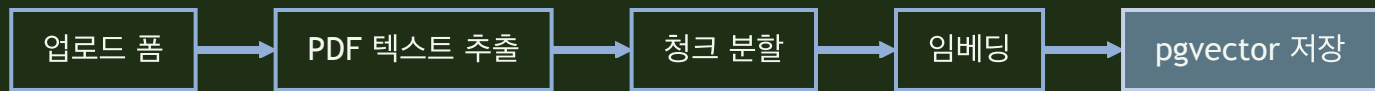
핵심 용어 (1/2)

용어	의미	예
문서 (Document)	업로드된 원본 자료	업로드한 보도자료 PDF
청크 (chunk)	문서를 나눈 조각	문단 하나 분량의 텍스트
임베딩 (embedding)	텍스트를 숫자로 표현한 값	[0.021, -0.153, ...]

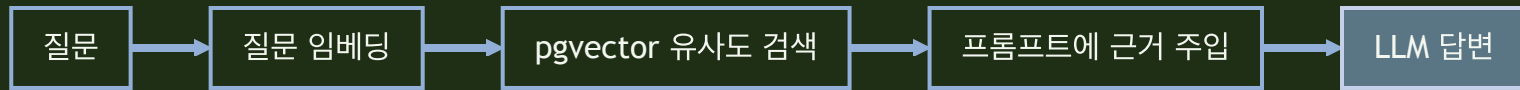
핵심 용어 (2/2)

용어	의미	예
벡터 스토어 (VectorStore)	임베딩을 저장하는 저장소	pgvector
유사도 검색	질문과 가까운 벡터를 찾는 검색	코사인 거리 기반 검색
메타데이터	벡터에 딸린 부가 정보	원본 파일명, 페이지 번호

PDF가 벡터로 저장되기까지



질문에 답하기까지



임베딩은 의미를 좌표로 바꾼다

- 벡터 변환: 문장을 고정 길이 실수 배열로 변환
- 거리와 의미: 의미가 비슷할수록 벡터 사이 거리가 가까움
- 검색 방식: 단어 일치가 아니라 의미 근접도로 검색
- 모델 일관성: 같은 모델로 만든 벡터끼리만 비교 가능

유사도를 재는 세 가지 방법

거리 방식	설명
코사인 거리	기본값, 벡터 방향만 비교해 길이 차이에 영향 없음
유클리드 거리	벡터 사이의 직선 거리
내적	두 벡터를 곱해 더한 값으로 비교

- 실습은 pgvector 기본값인 코사인 거리를 사용

차원 수를 먼저 정해야 하는 이유

- **인덱스 제약:** pgvector의 HNSW 인덱스는 최대 2,000차원까지만 인덱싱 지원
- **큰 벡터 불가:** 3072차원 같은 큰 벡터는 인덱스 생성 자체가 불가능
- **모델 선택 기준:** 2,000 이하를 지원하는 임베딩 모델 선택이 필수
- **변경 비용:** 모델을 바꾸면 테이블 컬럼과 인덱스를 재생성해야 하므로 초기에 신중히 결정

실습에 사용할 임베딩 모델

- 모델: Google `gemini-embedding-001` 사용
- 차원 축소: Matryoshka 축소로 1536차원 세팅
- 지원 기한: 종료 예정일 2028년 5월 14일, 2026년 8월 기준 사용 가능
- 대안 모델: `bge-m3` (1024차원)는 추후 Cloudflare Workers AI 구성 시 활용
- 향후 확장: `gemini-embedding-2` 는 이후 Image RAG(멀티모달) 차시에서 다룸

1536은 세 곳이 모두 같아야 한다

위치	설정 값
임베딩 모델 출력 차원	<code>dimensions: 1536</code>
DB 컬럼 타입	<code>vector(1536)</code>
벡터 스토어 설정	<code>spring.ai.vectorstore.pgvector.dimensions: 1536</code>

- 불일치 시 문제: 차원이 어긋나면 검색 자체가 불가능

pgvector 사용 준비 순서

단계	할 일
①	PostgreSQL JDBC·Spring Data JPA 및 Supabase DB 준비(이미 구축 가정)
②	Supabase Dashboard → Database → Extensions 이동
③	<code>vector</code> 확장 활성화 확인
④	Spring AI 기본 설정으로 스키마 자동 생성

자동으로 만들어지는 테이블

```
1  id          uuid          PK
2  content     text
3  metadata    json
4  embedding   vector(1536)
5  + HNSW 인덱스 (2000차원 이하 지원)
```

HNSW와 IVFFlat

인덱스	빌드 속도	메모리	검색 품질	특이사항
HNSW	느림	많음	좋음	기본값, 최대 2,000차원 한계
IVFFlat	빠름	적음	낮음	미리 학습(List) 필요

의존성 추가

```
1  implementation(platform("org.springframework.ai:spring-ai-bom:2.0.0"))
2  implementation("org.springframework.ai:spring-ai-starter-vector-store-pgvector")
3  implementation("org.springframework.ai:spring-ai-pdf-document-reader")
4  implementation("org.springframework.ai:spring-ai-starter-model-google-genai")
5  // Spring AI 2.0.0 GA, Spring Boot 4.0.x~4.1.x 지원
```

Chat 모델과 API 키 설정

```
1  spring:
2    ai:
3      google:
4        genai:
5          api-key: ${GOOGLE_AI_API_KEY}
6          chat:
7            model: ${app.ai.google.primary-model}
```

임베딩 모델과 차원 설정

```
1  spring:
2    ai:
3      google:
4        genai:
5          embedding:
6            text:
7              model: gemini-embedding-001
8              dimensions: 1536
```

벡터 스토어 설정

```
1  spring:
2    ai:
3      vectorstore:
4        pgvector:
5          initialize-schema: true
6          index-type: HNSW
7          distance-type: COSINE_DISTANCE
8          dimensions: 1536
9          table-name: vector_store
```

애플리케이션 커스텀 설정

```
1  app:
2    ai:
3      google:
4        primary-model: gemini-3.5-flash-lite
5        fallback-model: gemini-3.1-flash-lite
6      rag:
7        top-k: 5
8        similarity-threshold: 0.7
```

설정값을 타입 안전하게 묶기

```
1  @ConfigurationProperties(prefix = "app.ai")
2  public record AiProperties(Google google, Rag rag) {
3
4      public record Google(String primaryModel, String fallbackModel) {}
5
6      public record Rag(int topK, double similarityThreshold) {}
7  }
```

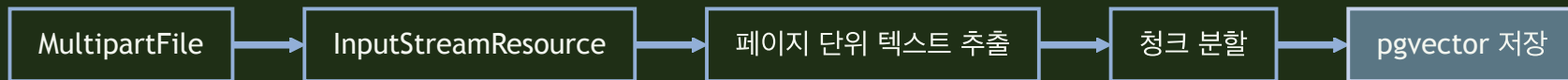
설정에서 조심할 것

- 타입 안전성: `@Value` 는 오타·미설정을 기동 시점에 걸러내기 어려워 `@ConfigurationProperties` 사용
- 기본값 유지: pgvector 설정은 기본값 유지가 가장 안정적
- 스키마 자동 생성: `initialize-schema: true` 는 개발·실습 단계에서만
- 테이블 삭제 위험: `remove-existing-vector-store-table: true` 는 데이터 손실 위험
- 키 관리: API 키는 소스와 저장소에 남기지 않고 환경 변수로 주입

Spring AI가 정리해 둔 ETL 3단계

인터페이스	역할	이번 실습 구현
<code>DocumentReader</code>	원본에서 <code>Document</code> 목록 읽기	<code>PagePdfDocumentReader</code>
<code>DocumentTransformer</code>	정제·청크 분할	<code>TokenTextSplitter</code>
<code>DocumentWriter</code>	저장소에 저장	<code>VectorStore</code> 구현체

업로드 요청이 처리되는 순서



PDF 업로드 폼

```
1 <form th:action="@{/rag/documents}" method="post" enctype="multipart/form-data">
2   <input type="file" name="file" accept="application/pdf" required />
3   <button type="submit">업로드</button>
4 </form>
```

업로드 파일을 스트림으로 읽기

```
1 Resource resource = new InputStreamResource(file.getInputStream());
2
3 PagePdfDocumentReader reader = new PagePdfDocumentReader(
4     resource,
5     PdfDocumentReaderConfig.builder().build()
6 );
```

청크로 나누어 벡터로 저장하기

```
1 // 검색에 적합한 크기(기본 800토큰)로 분할
2 List<Document> documents = new TokenTextSplitter().apply(reader.read());
3
4 // 임베딩 생성 후 pgvector에 저장
5 vectorStore.add(documents);
```

PagePdfDocumentReader가 못 읽는 PDF

- 기본 리더: 목차에 의존하지 않고 페이지 단위로 추출해 실습 기본값으로 사용
- 암호화·손상 PDF: 추출 자체가 실패
- 스캔 이미지 PDF: 텍스트가 거의 존재하지 않음
- 복잡한 레이아웃: 다단 구성·글자 매핑 오류 시 읽기 순서가 어긋남

업로드 파일 검증

- `accept` 속성: 브라우저 힌트일 뿐 보안 검증 수단 아님
- Content-Type 헤더: 클라이언트가 임의로 조작 가능해 신뢰 불가
- 서버 검증: 파일 크기 제한과 PDF 시그니처 확인 필수
- 거부 기준: 빈 파일·암호화·손상 파일은 저장 전에 차단

스캔 이미지 PDF 걸러내기

- 텍스트 부재: 이미지로만 구성된 PDF는 추출 텍스트가 거의 없음
- 저장 전 검증: 공백 제외 전체 길이와 페이지별 추출량 확인
- 기준 미달 처리: 벡터 저장을 중단하고 오류 반환
- 대안 경로: OCR·멀티모달 처리로 전환(405-2 Multi Modal RAG에서 다룸)

대용량 파일과 중복 업로드

- 동기 처리 부담: 읽기·분할·임베딩·저장을 한 번에 처리하면 응답 지연
- 처리 전략: 실습은 업로드 페이지 수 제한, 실무는 비동기 배치 처리
- 중복 업로드 문제: 같은 PDF 반복 업로드 시 중복 청크로 검색 결과 왜곡
- 중복 제거: 파일 해시·메타데이터로 검사 후 `vectorStore.delete( ... )` 로 재저장

임시 파일과 트랜잭션 경계

- `InputStreamResource`의 한계: 애플리케이션이 별도 임시 파일을 만들지 않게 할 뿐
- 서버 임시 저장: multipart 구현체가 임계값 초과 업로드를 서버 임시 디렉터리에 저장 가능
- 분리된 트랜잭션: 원본 파일 저장(객체 스토리지)과 벡터 DB 저장은 별개
- 정합성 관리: 한쪽만 성공하지 않도록 보정 로직 필요

유사도 검색 요청 만들기

```
1 SearchRequest request = SearchRequest.builder()
2     .query(question)
3     .topK(aiProperties.rag().topK())
4     .similarityThreshold(aiProperties.rag().similarityThreshold())
5     .build();
6
7 List<Document> results = vectorStore.similaritySearch(request);
```

메타데이터로 검색 범위 좁히기

```
1 SearchRequest.builder()  
2     .query(question)  
3     .topK(aiProperties.rag().topK())  
4     .filterExpression("fileName = '보도자료.pdf'")  
5     .build();
```

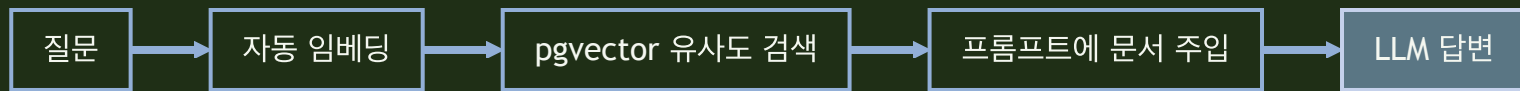
검색 범위 제한이 곧 보안

- 1단계 점검: 답변을 만들기 전에 `similaritySearch` 결과부터 확인
- 범위 제한: 특정 문서·작성자로 좁힐 때 `filterExpression` 사용
- 다중 사용자 환경: 소유자 필터를 필수로 적용
- 위험: 필터가 없으면 다른 사용자의 문서가 검색되어 유출로 이어짐

답변이 이상할 때 점검 순서

- 1단계: `similaritySearch` 결과에 원하는 문단이 걸리는지 확인
- 2단계: 걸리지 않으면 청크 크기 → `topK` → `similarityThreshold` 순으로 조정
- 기준값 주의: `0.7`은 정답이 아니라 시작점
- 분포 차이: 유사도 점수 분포는 임베딩 방식과 데이터에 따라 달라짐
- 결정 방법: 실제 질문-정답 평가셋으로 임곗값 결정

QuestionAnswerAdvisor가 대신 해주는 일



RAG 답변 생성

```
1 String answer = chatClient.prompt()  
2     .advisors(QuestionAnswerAdvisor.builder(vectorStore).build())  
3     .user(question)  
4     .call()  
5     .content();
```

근거 제시와 프롬프트 인젝션 대비

- 출처 표시: 검색된 `Document`의 메타데이터(파일명·페이지 번호)를 답변과 함께 화면에 표시
- 인젝션 위험: PDF 본문에 "기존 지시를 무시하라"는 문구가 숨어 있을 수 있음
- 방어 프롬프트: "검색된 문서만 근거로 답하고 없으면 모른다고 답하라"를 시스템 프롬프트에 명시
- 확장 옵션: 질문 재작성·멀티 쿼리 확장은 `RetrievalAugmentationAdvisor` (모듈러 RAG)로 확장

한도가 소진되면 무엇을 바꾸나

전환 방식	방법	주의
같은 공급자의 다른 모델	요청별 <code>ChatOptions</code> 전달	하나의 <code>ChatClient</code> 로 가능
다른 공급자	공급자별 <code>ChatClient</code> Bean을 만들고 라우터에서 선택	Google용 <code>ChatClient</code> 에 <code>OpenAiChatOptions</code> 를 넘기는 방식으로는 전환 불가

한도 소진일 때만 대체 모델 호출

```
1 public String answer(String question) {  
2     try {  
3         return callRag(question, aiProperties.google().primaryModel());  
4     }  
5     catch (RuntimeException exception) {  
6         if (!isModelQuotaExceeded(exception)) throw exception;  
7         return callRag(question, aiProperties.google().fallbackModel());  
8     }  
9 }
```

요청마다 모델 지정하기

```
1 private String callRag(String question, String model) {  
2     return chatClient.prompt()  
3         .options(GoogleGenAiChatOptions.builder().model(model).build())  
4         .advisors(QuestionAnswerAdvisor.builder(vectorStore).build())  
5         .user(question)  
6         .call()  
7         .content();  
8 }
```

실패 원인을 네 가지로 분류

분류	대표 상황	처리
RETRYABLE	순간적 RPM/TPM 제한· 타임아웃·5xx	Retry-After 또는 제한된 지수 백오프 후 같은 모델 재시도
SWITCH_MODEL	RPD·모델별 용량 소진	같은 공급자의 대체 모델을 한 번 호출
SWITCH_PROVIDER	공급자 장애·전체 한도 소진	다른 공급자의 ChatClient 호출
FAIL	인증·권한·잘못된 요청·안전 정책	fallback하지 않고 사용자 안내

fallback을 설계할 때의 원칙

- 오류 코드 확인: HTTP 429만 보지 말고 공급자 오류 코드까지 확인(OpenAI는 결제·사용량 한도 429가 섞여 있음)
- 재시도 계층 분리: 프레임워크 재시도와 애플리케이션 재시도를 겹치면 호출 횟수가 곱해지므로 한 계층만 담당
- 순환 경로 금지: `primary → fallback → primary` 같은 순환 경로를 만들지 않음
- 일관성 유지: fallback에도 같은 시스템 프롬프트·RAG Advisor·보안 필터를 그대로 적용
- 벡터는 그대로: 모델이 바뀌어도 임베딩 모델과 pgvector의 기존 벡터는 그대로

학습 요약 (1/2)

- RAG 구조: 질문과 관련된 청크만 찾아 프롬프트에 넣고 답을 생성
- ETL 파이프라인: `DocumentReader` → `DocumentTransformer` → `VectorStore`
- 차원 일치: `gemini-embedding-001` 을 1536차원으로 맞춰 HNSW 제한 충족
- PDF 추출: `spring-ai-pdf-document-reader` 로 업로드 스트림을 그대로 읽음

학습 요약 (2/2)

- 검색 점검: `similaritySearch`로 근거 문단이 걸리는지 먼저 확인
- 답변 생성: `QuestionAnswerAdvisor`로 검색·주입·생성을 한 번에 처리
- 보안 원칙: 소유자 필터와 방어 프롬프트로 유출·인젝션을 차단
- Fallback: 오류를 분류해 재시도·모델 전환·공급자 전환을 구분